Final Project

Multi-Platform Software Development for Real-World Problems (CSE)

Objective

Students will work in teams of 3 to 5 members to design and develop a **mobile application** that addresses a real-world problem.

Critical Requirement

Before any design or development begins, each team must conduct a **deep study and analysis** of the assigned problem. This includes:

- Understanding real user behavior
- Identifying informal workflows
- Capturing edge cases and real-life constraints

All findings must be **properly documented** and will be a major component of the evaluation.

Platform Requirement (Strict)

- **Mobile Application:** Mandatory
- **Web Application:** Optional
- **Recommended:** Flutter
- Alternatives: Kotlin, Swift, React Native
- Web-only systems, browser apps, or wrappers are strictly prohibited

Project Assignment

Each team will be assigned **one of the following problems**.
Assignment will be **random and fairly distributed**.

Problem 1: Smart Campus Companion

Deep Study Requirements

- Interview at least **10 students and 3 university staff members**
- Investigate:
    - How students access schedules, announcements, and services
    - Challenges in finding campus information
    - Communication gaps between departments and students
    - Common frustrations (missed updates, unclear schedules, navigation issues)

Core Features

- Student dashboard (schedule, alerts)
- Timetable and academic calendar
- Announcements & notifications
- Campus map and office directory
- Staff contact directory

Analysis Deliverable

- Document **3–5 real-life student experience scenarios**
- Identify inefficiencies in current campus information flow

Problem 2: Event Discovery & Ticket Booking System

Deep Study Requirements

- Interview at least **5 event organizers and 10 users**
- Investigate:
    - How events are currently promoted
    - Ticket booking challenges
    - Fraud or mismanagement issues
    - Problems in event awareness and participation

Core Features

- Event listing and search
- Event details (date, location, organizer)
- Ticket booking and QR-based validation
- Booking history tracking
- Notifications for upcoming events

Analysis Deliverable

- Identify **event participation gaps**
- Document **user journey from discovery to attendance**

Problem 3: Lost & Found Management System

Deep Study Requirements

- Interview at least **10 students or community members**
- Investigate:
    - How lost items are currently handled
    - Common places/items frequently lost
    - Recovery challenges
    - Trust and verification issues

Core Features

- Post lost/found items with images
- Search and filtering system
- Contact/messaging between users
- Status tracking (lost → found → returned)

Analysis Deliverable

- Document **real cases of lost items**
- Identify **failure points in recovery processes**

Problem 4: Job & Internship Finder Platform

Deep Study Requirements

- Survey at least **10 students and 3 employers/recruiters**
- Investigate:
    - How students currently find jobs/internships
    - Challenges in applying and tracking applications
    - Employer difficulties in finding suitable candidates
    - Communication gaps

Core Features

- Job/internship listings
- Search and filtering
- Application submission system
- Resume (CV) upload
- Employer job posting interface

Analysis Deliverable

- Map current job search workflow
- Highlight at least **3 major inefficiencies**

Problem 5: Complaint & Feedback Management System

Deep Study Requirements

- Interview at least **8–10 users and 2 administrators**
- Investigate:
    - How complaints are currently submitted
    - Delays in response and resolution
    - Lack of transparency in status tracking
    - Common dissatisfaction points

Core Features

- Complaint submission with attachments
- Categorization system
- Status tracking (pending → in progress → resolved)
- Admin response dashboard
- Notifications and updates

Analysis Deliverable

- Document **real complaint scenarios**
- Identify **gaps in accountability and tracking**

Mandatory SDLC Phases

Phase	**Description**
Deep Study & Analysis	User research, workflows, real-world scenarios
Requirement Specification	Functional & non-functional requirements
Design	Architecture, database, UI/UX
Development	Functional mobile application
Testing	Unit, integration, and UAT

Complete Deliverables (Mandatory)

1. Analysis & Requirements Document

- User interviews / surveys
- Identified pain points
- Real-world scenarios (minimum 3–5)
- As-Is workflow
- Functional requirements
- Non-functional requirements
- Use cases / user stories

2. Design Documents

- System architecture diagram
- Database schema (ERD)
- UI/UX wireframes or mockups
- API design

3. Implementation

- Fully functional mobile application
- Clean and modular code
- Version control (Git recommended)

4. Testing Documentation

- Test cases based on real scenarios
- Unit & integration testing
- User Acceptance Testing (UAT)

5. Final Presentation

- Problem analysis findings
- Design explanation
- Live demo
- Testing results
- Team contribution

Innovation & Additional Features

Students are encouraged to go beyond minimum requirements:

- Offline-first functionality
- AI-based features (recommendation, automation)
- Analytics dashboards
- Security improvements
- Integration (SMS, APIs, notifications)
- Advanced UI/UX
- Local language integration and
- Others

Evaluation & Presentation

Students must present:

- Analysis and research
- System design
- Working application
- Testing results
- Contribution breakdown

Important Note

Students may use any tools, including **AI tools and LLMs**, during development. However, if two or more projects are found to be **significantly similar**, their evaluation score will be reduced by **50%**.